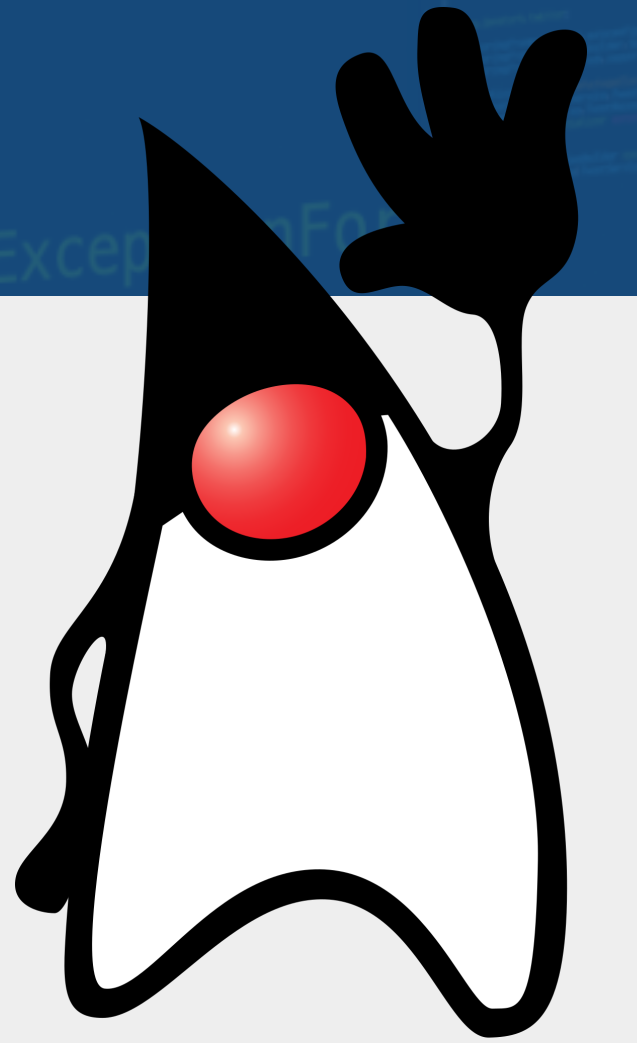


Java Programlama Dili

21. Bölüm

Kod Organizasyonu

Akın Kaldıroğlu



Amaç



- Bu bölümde kod organizasyonu ele alınacaktır, `package` ve `import` cümleleri incelenecektir
- Bu bölümde anlatımlar sınıf üzerinden yapılmasına rağmen anlatılanlar ileride öğreneceğiniz arayüz (interface) vb. Java'daki diğer karmaşık tipler için de geçerlidir



- **Paket**
 - İsim Uzayı
 - Kısa Kaynak Dosyalarında Paket
 - Tam İsim
- **Paketler Arası Erişim: import**
 - Kısa Kaynak Dosyalarında Import
 - Paket
- İsim Çakışması
- static import
- Java Kütüphanesinin Paketleri
- **Paket ve Dizin**
- **Derleme**
- **Ödevler**
- **Bundan Sonra?**

Paket



Kod Organizasyonu



- Programlama dillerinde kodu organize etmek için module, package, namespace vb. isimlerle oluşturulan gruplayıcı yapılar vardır
- Kod organizasyonunun amacı:
 - Kodu mantıksal, fonksiyonel hatta fiziksel açıdan kategorize etmek
 - Kodun farklı kısımları arasında erişim kuralları koymak ve
 - İsim uzayı oluşturup çakışmaları önlemektir

Paketler - I



- Java'da kod organizasyonu temelde sınıf vb. tipler üzerinden yapılır ve paketler yaratılır
- Paket **package** anahtar kelimesi ve paketin ismiyle oluşturulur (package declaration)
- **package** cümlesi kaynak dosyasında bir kere ifade edilir ve dosyadaki tüm sınıflar o pakete ait olur

```
package myPackage;  
ya da  
package org.javaturk.jp.ch04.car;
```

Paketler - II



- Dolayısıyla bir Java kaynak dosyasında ancak bir tane paket ifadesi olabilir
- Her tipin sadece bir paketi olabilir
- Paket zorunlu değildir
- Paket ismi, geçerli Java belirteçlerinden (identifier) oluşmalıdır
- Bir kaynak dosyasında birden fazla tip varsa hepsi aynı pakete dahil olur

Örnek



- Örnekte `MyClass.java` dosyası içinde aynı pakete ait olacak şekilde tanımlanmış 3 sınıf vardır
- Tabii olarak bu sınıflardan sadece bir tanesi `public`'tir

```
package org.javaturk.jp.ch21;  
  
public class MyClass {  
  
}  
  
class HerClass{  
  
}  
  
class HisClass{  
  
}
```



- `MyClass`

Paketler - III



- **package** cümlesi, Java kaynak kodunda yer alan ilk cümle olmalıdır, dolayısıyla **package** cümlesinden önce sadece boşluk ve yorum satırları olabilir
- Paket yapısı, `class` dosyaları için geçerlidir, `java` kaynak kodları için değil!
- Fakat genelde kolaylık olması için kaynak kodları da `class` dosyalarına paralel bir şekilde paketlere göre düzenlenir; örneğin IDE'ler Java kaynak kod dosyalarını da bu şekilde düzenlerler

Paketler - IV



- Paketler genelde iç içe geçerek katmanlı bir yapı (hierarchy) oluşturur
- İç içe paketler birbirlerinden `.` ile ayrılır
- Bu örnekte `org`, **en üst seviye pakettir (top level package)**

```
package org.javaturk.jp.ch04.car;
```

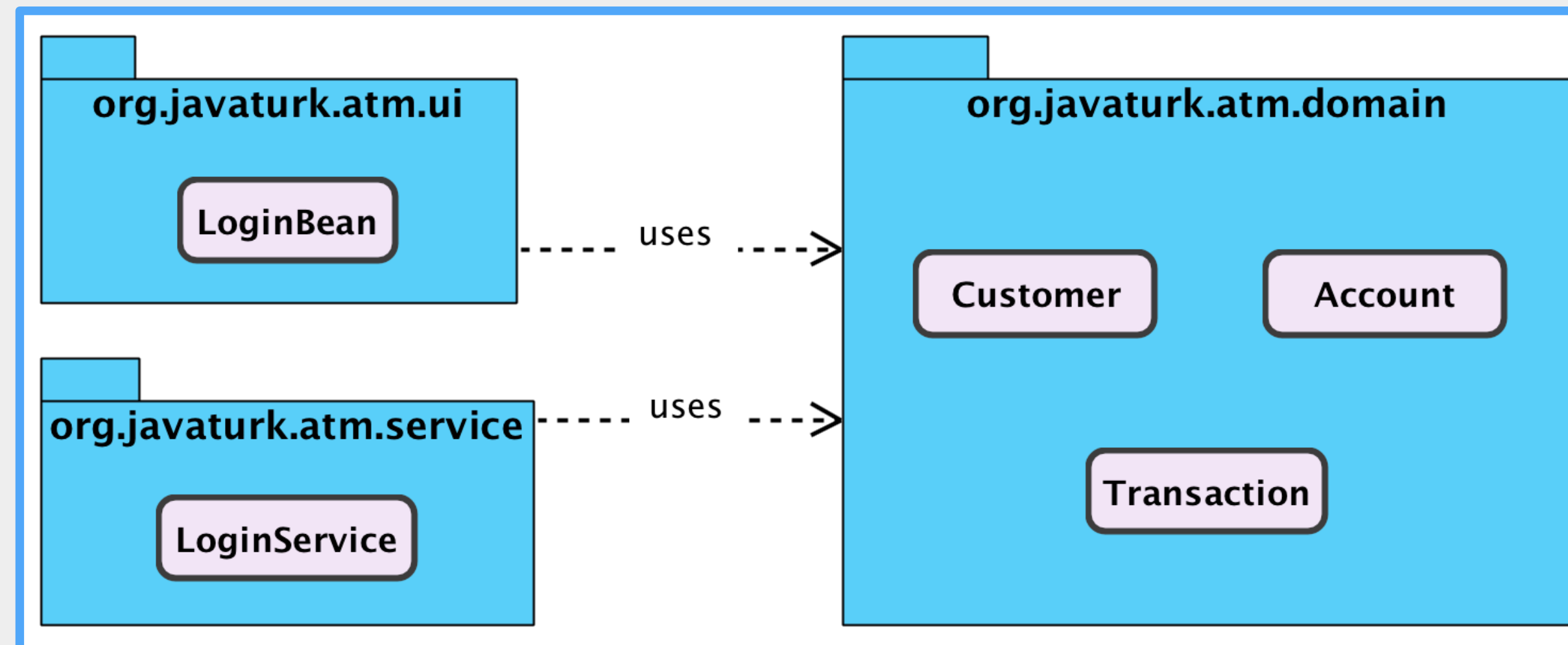
- Böylece en üst seviyedeki paketten başlayarak alt paketler içinde tipler, anlamlı kategorizasyona ve buna karşı gelen fiziksel organizasyona sahip olur

Paketler - V



- Java'da paketler, işletim sisteminin dizinlerine (directory, folder) karşılık gelir
- Bazı dillerde bu bir zorunluluk değildir, paket yapısı sadece kavramsal bir grupta olarak kalır
- Dolayısıyla iç içe paketler de iç içe geçmiş dizinlerle gerçekleştirilir
- Aynı paketteki yapılar aynı dizinde bulunur

Örnek - I



```
package org.javaturk.atm.domain; // Customer.java
package org.javaturk.atm.domain; // Account.java
package org.javaturk.atm.domain; // Transaction.java

package org.javaturk.atm.ui; // LoginBean.java

package org.javaturk.atm.service; // LoginService.java
```

Örnek - II



- Her `public` sınıfın kendi kaynak kod dosyasında tanımlanması gerektiğinden, aşağıdaki sınıfların ayrı paket tanımları olacaktır

```
package org.javaturk.atm.domain;  
  
public class Customer{  
    ...  
}
```

```
package org.javaturk.atm.service;  
  
public class LoginService{  
    ...  
}
```

```
package org.javaturk.atm.ui;  
  
public class LoginBean{  
    ...  
}
```

Paketlere Karar Vermek



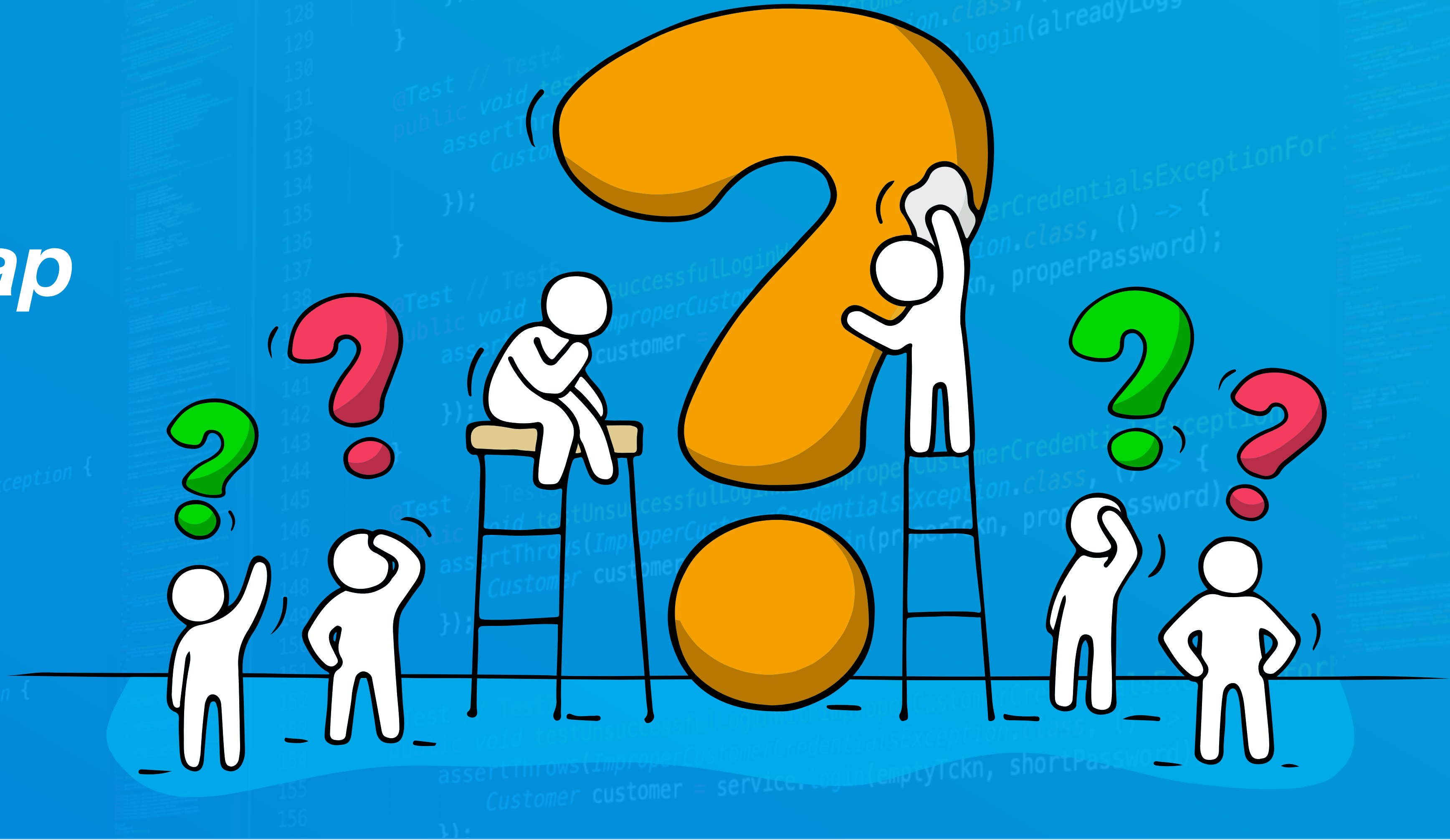
- Kategorizasyon kriteri olarak fonksiyonellik seçilebilir
- Aynı amaca yönelik tipler aynı pakette olur; örneğin:
 - Grafik arayüz, entegrasyon ya da veri gibi katmanlar (layer)
 - Müşteri ve banka hesabı gibi iş alanı (domain) grupları
 - Loglama, exception ya da transaction gibi farklı ilgi alanları (aspects)
- Sıklıkla aynı paketteki yapılar birlikte değişir ve yeni sürüme geçer

Varsayılan ya da İsimsiz Paket



- Eğer bir Java kaynak kodunda paket tanımı yoksa bu durumda o kaynak kodundaki tipler **varsayılan pakete (default package)** ya da **isimsiz pakete (unnamed package)** sahip olur
- İsimsiz paketlere izin verilmesinin sebebi denemeler için sağladığı kolaylıktır
- Aslolan paket kullanmaktır

Soru ve Cevap Zamanı!





Kısa Kaynak Dosyalarında Paket

Kısa Kaynak Dosyaları - I



- Daha önce, 4. bölümde detaylıca ele alınmıştı: Java 25'de **Compact Source Files and Instance Main Methods (Kısa Kaynak Dosyaları ve Nesne Main Metotları)** adı ile kod yazmayı kolaylaştırıcı ve kısaltıcı özellikler geldi
- Bu özellikle, sınıf olmadan sadece `main` metot kullanarak geçerli Java kodu oluşturmak mümkün oldu; bu şekilde oluşturulmuş kaynak koda *compact source file (kısa kaynak dosyası)* adı verilir

```
void main() {  
    IO.println("Hello");  
}
```

Kısa Kaynak Dosyaları - II



- Bu şekilde, sınıf oluşturmada kod yazıldığında kaynak kodda bulunmasa bile derleyici arka planda, `class` dosyasında görünecek şekilde, bir sınıf yaratır ki bu sınıfa *implicitly declared top-level class* (örtük olarak tanıtılmış üst düzey sınıf) denir

```
//CompactGreeting.java
//package org.javaturk.jp.ch21.compact;
import org.javaturk.jp.ch21.compact.Hello;
import org.javaturk.jp.ch21.compact.Selam;

Random random = new Random();
String start = "Compact Greeting";
Selam s = new Selam();
Hello h = new Hello();

void main() {
    System.out.println(start);
    saySalam("Fatma");
    sayHello("John");
}

void saySalam(String whom) {
    ...
}

void sayHello(String whom) {
    ...
}
```

Kısa Kaynak Dosyaları ve Paket



- Bu özellikle gelen kısa kaynak kod dosyaları bir pakete sahip olamazlar
- Kısa kaynak kod dosyalarına **package** cümlesi yazmak bir derleme hatasıdır

```
//CompactGreeting.java
//package org.javaturk.jp.ch21.compact;
import org.javaturk.jp.ch21.compact.Hello;
import org.javaturk.jp.ch21.compact.Selam;

Random random = new Random();
String start = "Compact Greeting";
Selam s = new Selam();
Hello h = new Hello();

void main() {
    System.out.println(start);
    saySalam("Fatma");
    sayHello("John");
}

void saySalam(String whom) {
    ...
}

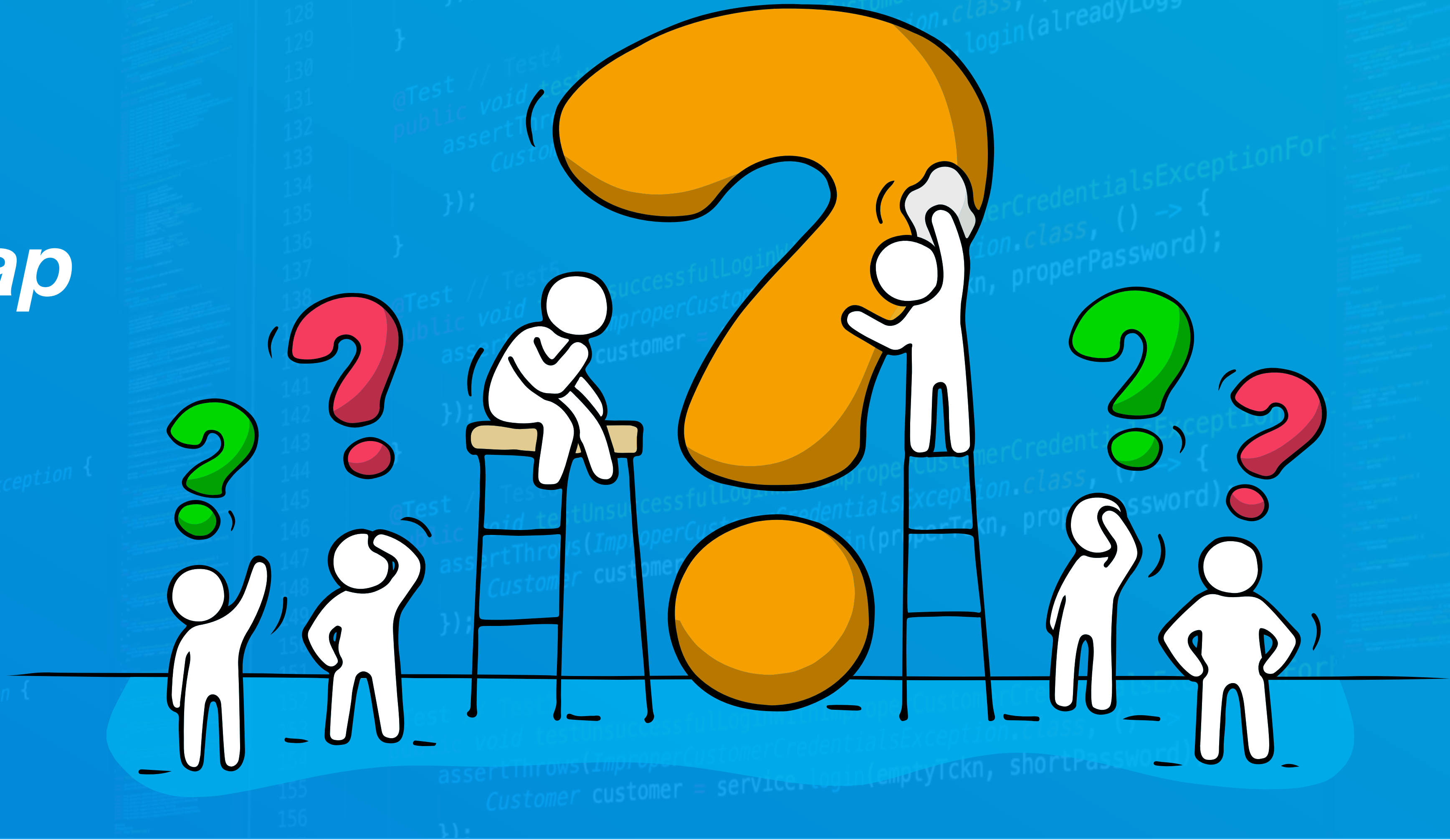
void sayHello(String whom) {
    ...
}
```

CompactGreeting



- `compact.CompactGreeting`

Soru ve Cevap Zamanı!





İsim Uzayı

İsim Uzayı - I



- Paketin bir diğer amacı da bir **isim uzayı (namespace)** oluşturarak muhtemel çakışmaları önlemektir
- Bir projede yazılan birkaç tane farklı **Printer** ya da **Date** sınıfının, birbirleriyle ya da dışarıdan gelen bileşenlerdeki (third party components) aynı isimdeki sınıflarla karışmasını önlemenin yolu, her bir **Printer** ya da **Date** sınıfının ayrı pakette olmasıdır

İsim Uzayı - II



- Peki paketler de çakışmaz mı?
- Kurumlar ya da kişiler kendi internet alan (domain) adresini tersinden kullanarak paketler oluşturulduğunda çakışma olmayacaktır

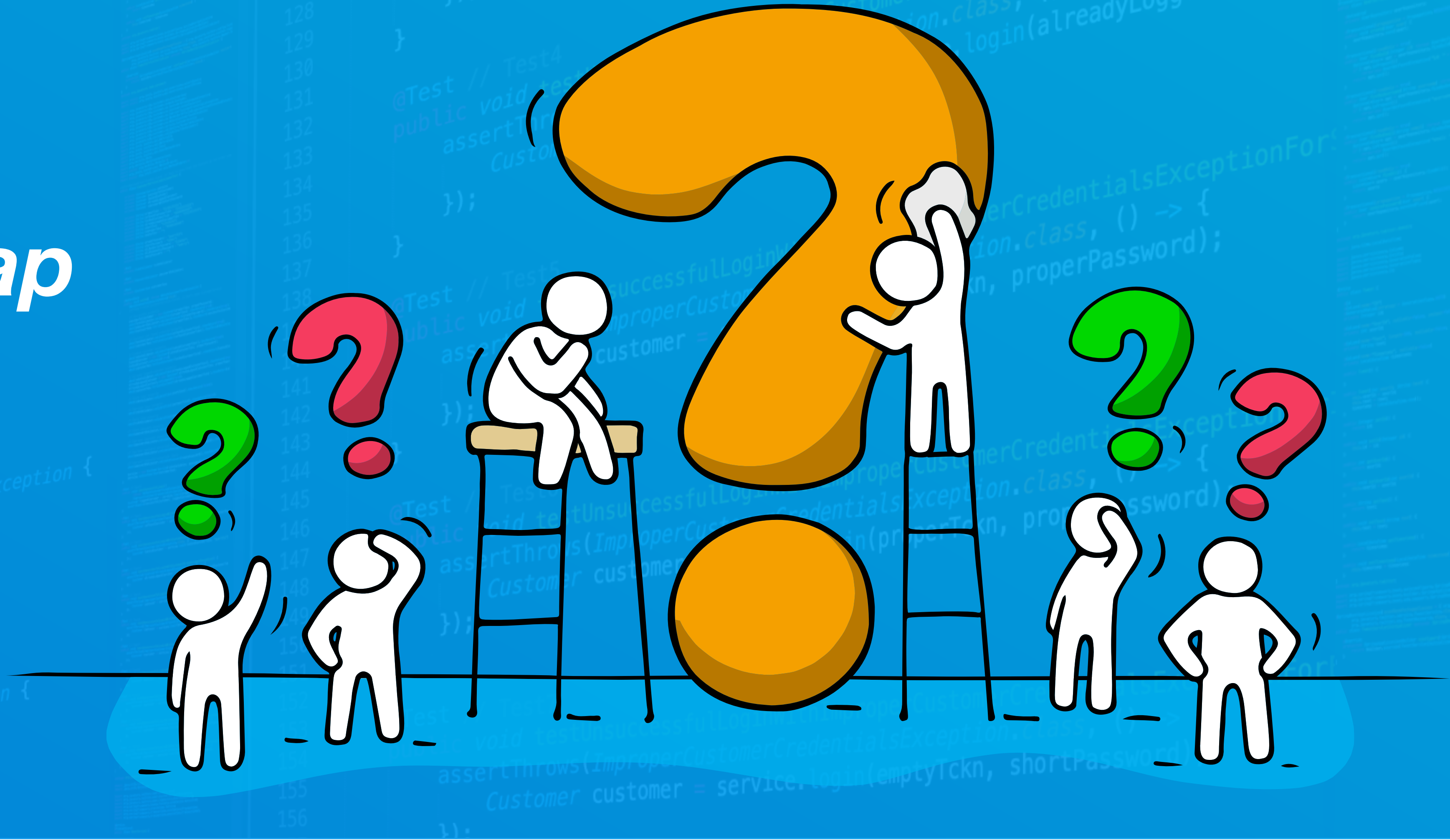
```
com.myCompany.myProject.mySubPackage  
org.javaturk.jp  
org.javaturk.oop  
org.javaturk.fp  
org.javaturk.cc  
org.javaturk.dp
```

```
// For an ATM project  
org.javaturk.atm.domain  
org.javaturk.atm.ui  
org.javaturk.atm.service  
  
dev.kaldiroglu.jp.ch10
```

```
com.google.cloud.language  
com.facebook.business  
tr.com.selsoft.crm
```

- Dolayısıyla siz de Java kodlarınız için kendinize bir isim uzayı belirleyin ve proje, ödev, deneme vb. amaçlarla yazdığınız kodlarda kullanın

Soru ve Cevap Zamanı!





Tam İsim

Tam İsim - I



- Bir sınıfın paketsiz olan ismine **basit isim (simple name)** denir
- Bir pakete sahip olan bir sınıfın ismi artık **paket.sınıfismi** olarak değişmiştir
- Buna **tam isim (fully qualified name)** denir

```
package org.javaturk.atm.ui;  
  
public class LoginBean{  
    ...  
}
```

```
LoginBean // Simple name  
org.javaturk.atm.ui.LoginBean // Fully qualified name
```

Tam İsim - II



- Tam isim, kaynak kod için değil, sınıfın derlenmiş yani `class` dosyasındaki kodu için geçerlidir
- Kaynak kodun ismi, dosyadaki `public` sınıfın ismidir
 - Eğer sınıf `public` değilse kaynak kodun ismi dosyanın ismidir
- Tam isim kavramsaldır fakat fiziksel olan dizinleri de belirler
 - Çünkü paket ismi İS'ndeki dizinlere karşılık gelir

Tam İsim - III



- Bundan dolayı paketi olan bir sınıfı çalıştırılmak için JVM'e (java komutuna) geçilirken, artık sınıfın tam isminin kullanılması gereklidir
- Ama derleme için değişen bir şey yoktur çünkü derlenirken dosya ismi kullanılır

```
package org.javaturk.atm.ui;           // LoginBean.java
javac LoginBean.java                   // Compiling
java org.javaturk.atm.ui.LoginBean     // Running

package org.javaturk.atm.service;      // LoginService.java
javac LoginService.java                // Compiling
java org.javaturk.atm.service.LoginService // Running
```



Uygulama

Uygulama

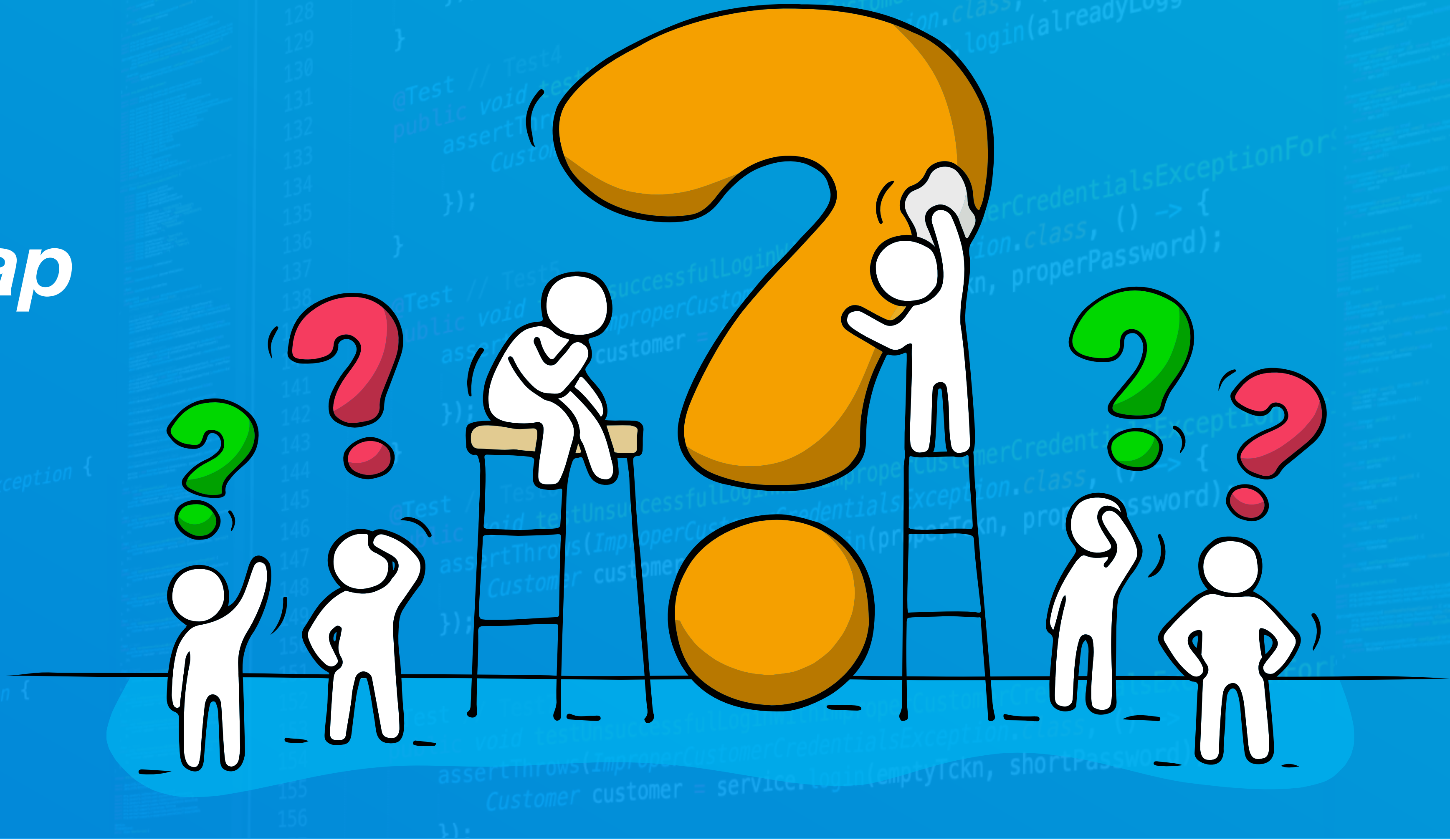


- Yandaki `Selam` sınıfına `a.b` gibi bir paket ekleyip komut satırında derleyin ve `class` dosyasının oluştuğunu gözlemleyin
- Bu sınıfın nesnesini oluşturacak şekilde bir `main` metot vs. yazmayın çünkü henüz çalıştırabilmek için farklı bilgilere ihtiyacımız var

```
public class Selam{
    String world = "millet";

    public String selamSoyle(String kime) {
        String cumle;
        if (!kime.isEmpty())
            cumle = "Selam " + kime + " :)";
        else
            cumle = "Selam " + world + " :)";
        return cumle;
    }
}
```

Soru ve Cevap Zamanı!



Paketler Arası Erişim: import



Paketler Arası Erişim



- Aynı paketlerdeki sınıflar birbirlerine basit isimleriyle erişebilir; tam isim kullanmaya gerek yoktur
- Farklı paketlerdeki sınıflar birbirlerine ancak tam isimleriyle erişebilir
- Ancak tam isimle erişimin problemlili olduğu açıktır çünkü tam isim uzundur

```
package org.javaturk.jp.ch21.packageX;  
  
public class ClassX {}
```

```
package org.javaturk.jp.ch21.packageA;  
  
class ClassA {  
    org.javaturk.jp.ch21.packageX.ClassX x;  
  
    ClassA(org.javaturk.jp.ch21.packageX.ClassX x){  
        this.x = x;  
    }  
}
```

ClassA



- `packageA.ClassA`

import - I



- Tam ismi kullanmanın alternatifi **import** cümlesidir
- Farklı paketlerdeki yapılar birbirlerine basit isimleriyle erişmek istediklerinde **import** kullanırlar
- Bir sınıfı bir kaynak koduna **import** etmek, o sınıfı, tam ismi yerine basit ismiyle kullanılabilmek için kaynak koduna dahil etmek demektir

import - II



- **import** cümlesinde, kaynak dosyasındaki sınıfların erişmek istediği ama paketi farklı olan tipin tam ismi yer alır
- **import** cümlesi zorunlu değildir, ancak farklı paketteki bir tip basit ismiyle kullanılmak istendiğinde gerekli olur
- Birden fazla tipe erişim gerekliyse her tip için ayrı bir **import** cümlesi yazılır

```
import org.javaturk.jp.ch04.car.Car;
```

```
import org.javaturk.jp.ch04.car.Car;  
import java.util.List;  
import org.springframework.beans.factory.BeanFactory;
```

import - III



- Bir kaynak kod dosyasındaki tüm sınıflar, **import** edilen sınıfı, sadece basit ismiyle kullanılabilir
- Tam ismi kullanmaya artık gerek yoktur
- Bir kaynak kod dosyasında **import** edilen sınıflar, o dosyadaki tüm sınıflar tarafından basit isimleriyle kullanılabilir

```
package org.javaturk.jp.ch21.packageA;

import org.javaturk.jp.ch21.packageX.ClassX;

class ClassA {
    ClassX x;

    ClassA(ClassX x){
        this.x = x;
    }
}

class ClassB {
    ClassX x;

    void setX(ClassX x){
        this.x = x;
    }
}
```

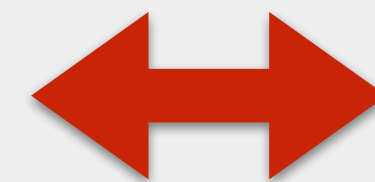
import - IV



- **import** kullanarak basit isimle erişimin sağladığı kolaylık!

```
package org.javaturk.jp.ch21.packageX;  
  
public class ClassX {}
```

```
package org.javaturk.jp.ch21.packageA;  
  
class ClassA {  
    org.javaturk.jp.ch21.packageX.ClassX x;  
  
    ClassA(org.javaturk.jp.ch21.packageX.ClassX x){  
        this.x = x;  
    }  
}
```



```
package org.javaturk.jp.ch21.packageA;  
  
import org.javaturk.jp.ch21.packageX.ClassX;  
  
class ClassA {  
    ClassX x;  
  
    ClassA(ClassX x){  
        this.x = x;  
    }  
}
```

ClassB



- `packageB.ClassB`
- `ClassB.java` kaynak kod dosyasındaki tüm tiplerin **`import`** edilen tiplere ulaştığına dikkat edin

import - V



- Java kaynak kodunda **import** cümlesi, varsa **package** cümlesinden sonra ve sınıflardan önce gelir
- **package** yoksa, **import** cümlesi ilk cümle olur
 - Öncesinde sadece boşluk ve yorum olabilir
- **package** cümlesi varsa onunla **import** cümlesi arasına sadece başka bir **import** cümlesi girebilir
- **import** cümlelerinin arasına başka hiçbir kod giremez

import - VI



- Bir Java kaynak kod dosyasında her şey muhakkak bir sınıf içinde olmalıdır
- **Everything goes into a class!**
- Bunun iki istisnası, sınıf hakkında olan **package** cümlesi ile basit ismiyle erişilecek sınıfları dahil eden **import** cümleleridir
- **package** ve **import** cümleleri muhakkak sınıflardan önce gelir

```
<package cümlesi>

<import cümlesi>*

public class MyClass{
    ...
}

class <Sınıfİsmi2>{
    ...
}

...
class <SınıfİsmiN>{
    ...
}
```

Dosya ismi: MyClass.java

import - VII



- Kısa kaynak kod dosyalarının da, sınıf tanımlamadan kod yazmaya izin verdiği için, bunun bir istisnası olduğunu öğrendik

import - VIII



- Eğer aynı paketten birden fazla tipe erişmeye ihtiyaç varsa, bu tipler tek tek `import` edilebileceği gibi, paketin içindeki tiplerin hepsi `*` ile hep birlikte `import` edilebilir

```
package org.javaturk.jp.ch21.packageB;
import org.javaturk.jp.ch21.packageX.ClassX;
import org.javaturk.jp.ch21.packageX.ClassZ;
// ya da
import org.javaturk.jp.ch21.packageX.*;

public class ClassB {
    ClassX x;
    ClassZ z;
    ...
}
```

import - IX



- `import`'ta ya `*` ile tüm paket ismi ya da tek tek ulaşılacak tipler yazılır
- İki gösterimin kısa yazma ile tek tek yazma dışında farkı, birbirlerine performans vb. üstünlükleri, yoktur
- İkisi de ihtiyaç olduğunda sınıfların bulunabilecek yerini gösterir, koda dahil edilen sınıflar ancak ihtiyaç olduğunda yüklenir, kullanılmazlarsa yüklenmezler

```
import org.javaturk.jp.ch21.packaging.packageX.ClassX;  
import org.javaturk.jp.ch21.packaging.packageX.ClassY;  
import org.javaturk.jp.ch21.packaging.packageX.ClassZ;  
// ya da  
import org.javaturk.jp.ch21.packaging.packageX.*;
```

import - X



- * ile alt paketler `import` edilemez aksi takdirde örneğin `org.*` ile projedeki her şey `import` edilirdi!
- Ayrıca bir paket parçalı olarak `import` edilemez ya da paket değil ancak içindikiler `import` edilebilir

```
package org.javaturk.jp.ch21;
import org.*; // Imports only types under "org"

public class ImportingAnotherClass{

    AnotherClass anotherClass; // OK
    javaturk.jp.ch21.MyClass myClass = new javaturk.jp.ch21.MyClass(); //Error
}
```

```
package org;

public class AnotherClass{
    ...
}
```

```
package org.javaturk.jp.ch21;

public class MyClass{
    ...
}
```

ImportingAnotherClass



- `ImportingAnotherClass`

import - XI



- Ayrıca `import` işlemi geçişli değildir yani bir sınıf ancak doğrudan `import` ettiği yapıları basit ismiyle kullanabilir, `import` ettiği sınıfın `import` ettiği yapılara basit isimleriyle ulaşamaz
- Yani **A**, **B**'yi, **B**, **C**'yi import ederse, **A**'nın **C**'ye ulaşması için **C**'yi import etmesi gerekir
- Tabii olarak **A**, **B** ve **C**'nin farklı paketlerde olduğu varsayılmıştır

ClassB



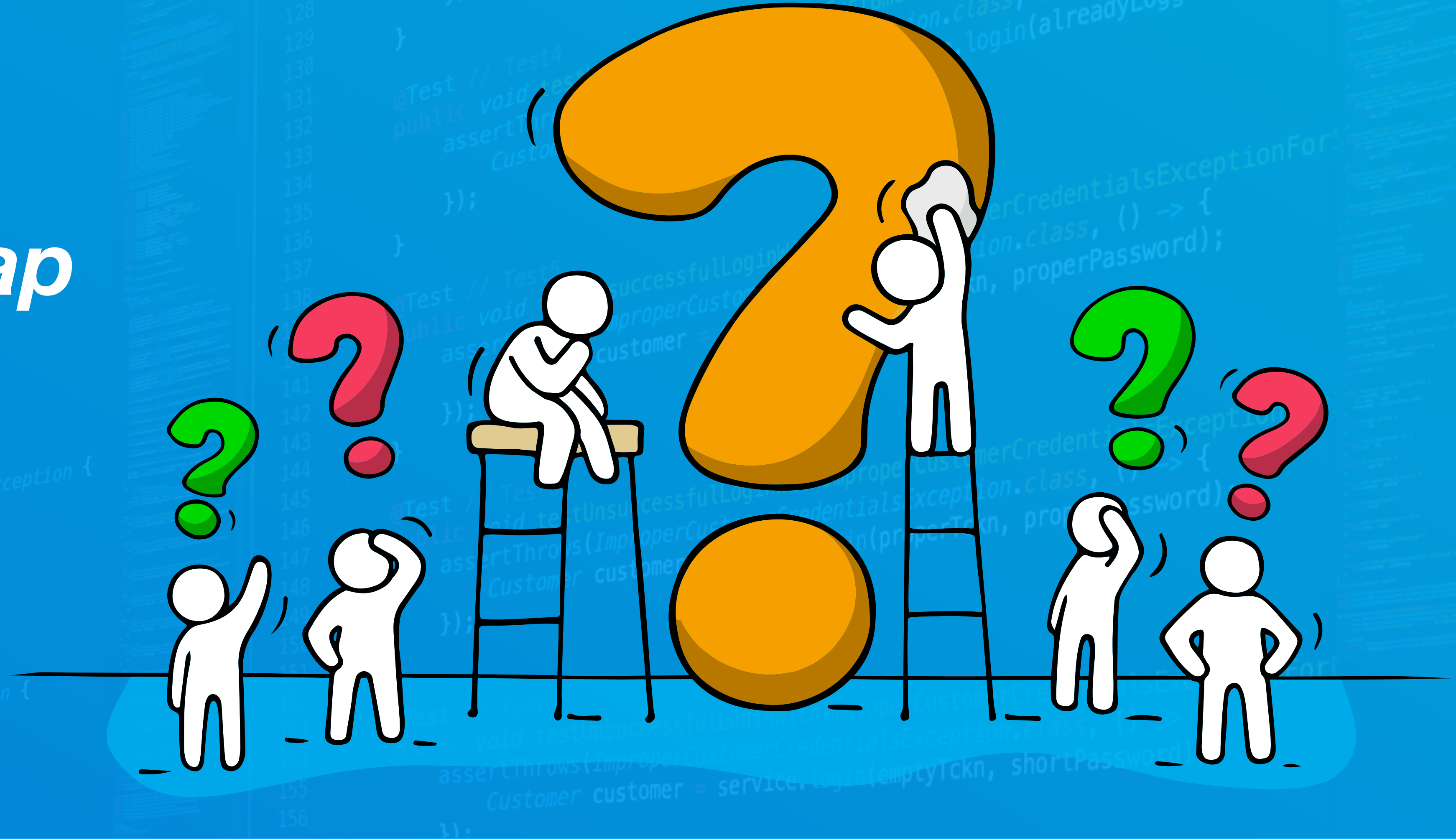
- `packageB.ClassB`
- `ClassB.java` kaynak kod dosyasındaki tüm tiplerin **`import`** edilen tiplere ulaştığına dikkat edin

Varsayılan Paketten Import



- İsimsiz yani varsayılan paket kullanmanın negatif tarafı, isimsiz paketten bir tip `import` edilemeyeceği gerçeğidir
- Dolayısıyla çok basit, deneme cinsinden kodlarınız dışında daima paket kullanın
- Hatta denemelerinizi bile paketler altında organize edin, ileride bulması kolay olur

Soru ve Cevap Zamanı!





Kısa Kaynak Dosyalarında Import

Kısa Kaynak Dosyaları ve Import - I



- Kısa kaynak kod dosyalarının bir pakete sahip olamayacaklarını az önce konuştuk
- Ama kısa kaynak kod dosyaları başka sınıfları **import** edebilirler
- Kısa kaynak kod dosyaları bir pakete sahip olmadıklarından aynı dizinde olsa bile diğer sınıflara ancak **import** ile ulaşabilirler

```
//CompactGreeting.java
import org.javaturk.jp.ch21.compact.Hello;
import org.javaturk.jp.ch21.compact.Selam;

Random random = new Random();
String start = "Compact Greeting";
Selam s = new Selam();
Hello h = new Hello();

void main() {
    System.out.println(start);
    saySelam("Fatma");
    sayHello("John");
}

void saySelam(String whom) {
    ...
}

void sayHello(String whom) {
    ...
}
```

Kısa Kaynak Dosyaları ve Import - II



- *Kısa kaynak kod dosyaları, aynı dizinde olsa bile diğer sınıflara ancak **import** ile ulaşabilirler* dedik
- Ayrıca bu şekilde kısa kaynak kod dosyalarına import edilen sınıflar ve üyeleri muhakkak **public** olmalıdır
- Bir erişim niteleyici olan **public** anahtar kelimesini ileriki bir bölümde ele alacağız

```
//CompactGreeting.java
import org.javaturk.jp.ch21.compact.Hello;
import org.javaturk.jp.ch21.compact.Selam;

Random random = new Random();
String start = "Compact Greeting";
Selam s = new Selam();
Hello h = new Hello();
...
```

```
package org.javaturk.jp.ch21.compact;

public class Hello {
    String greeting = "Hello ";

    public String sayHello(String whom) {
        return greeting + whom + " :~)";
    }
}
```

Kısa Kaynak Dosyaları ve Import - II



- Ayrıca kısa kaynak dosyalarının derlenmesiyle oluşturulan örtük sınıflara (implicit classes) başka bir sınıftan da ulaşılamaz, **import** edilemez
- Dolayısıyla kısa kaynak kod dosyaları, kaynak kod dosyalarındaki pek çok sınıftan oluşan bir programın ancak giriş noktası olarak kullanılabilirler

```
//CompactGreeting.java
import org.javaturk.jp.ch21.compact.Hello;
import org.javaturk.jp.ch21.compact.Selam;

Random random = new Random();
String start = "Compact Greeting";
Selam s = new Selam();
Hello h = new Hello();

void main() {
    System.out.println(start);
    saySelam("Fatma");
    sayHello("John");
}

void saySelam(String whom) {
    ...
}

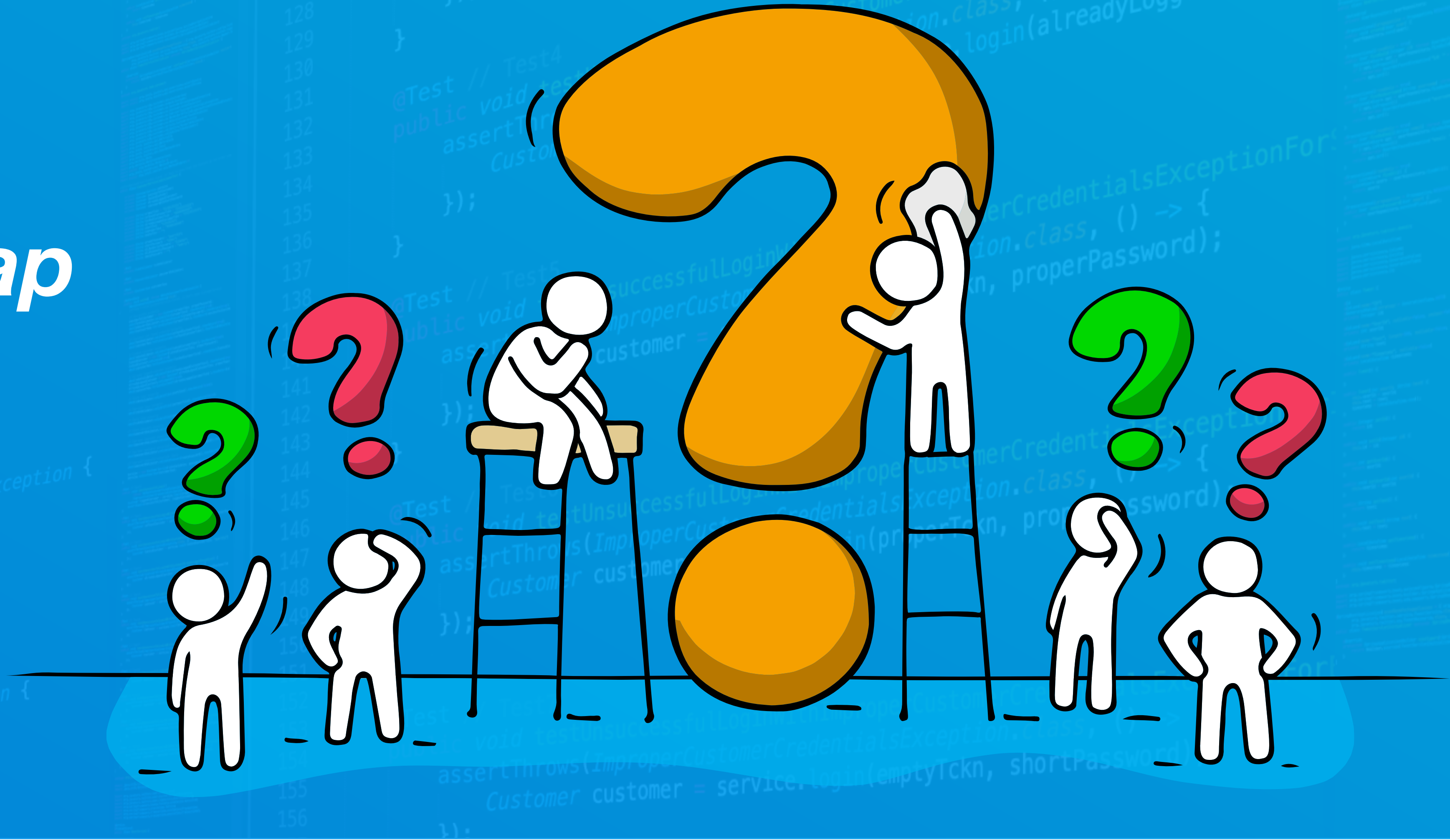
void sayHello(String whom) {
    ...
}
```

CompactGreeting



- `compact.CompactGreeting`
- Aynı dizindeki `Hello` ve `Selam` sınıflarının ve metotlarının `public` niteleyicilerini kaldırın ve sonucu gözlemleyin

Soru ve Cevap Zamanı!





İsim Çakışması

Aynı İsimli Sınıfları Dahil Etme - I



- Farklı paketlerde olup da aynı isme sahip olan sınıflar bir kaynak kodda nasıl kullanılır?
- Bu durumda aynı isme sahip olan bu sınıflardan sadece birisi **import** ile koda dahil edilir, diğer sınıflar tam ismiyle kullanılır
- Ya da uzun yazılmış ifadelerde hangisinin basit ismiyle kullanıldığı karışıklık çıkarıyorsa, hiçbirini dahil etmeyip hepsi daima tam ismiyle de kullanılabilir

Aynı İsimli Sınıfları Dahil Etme - II

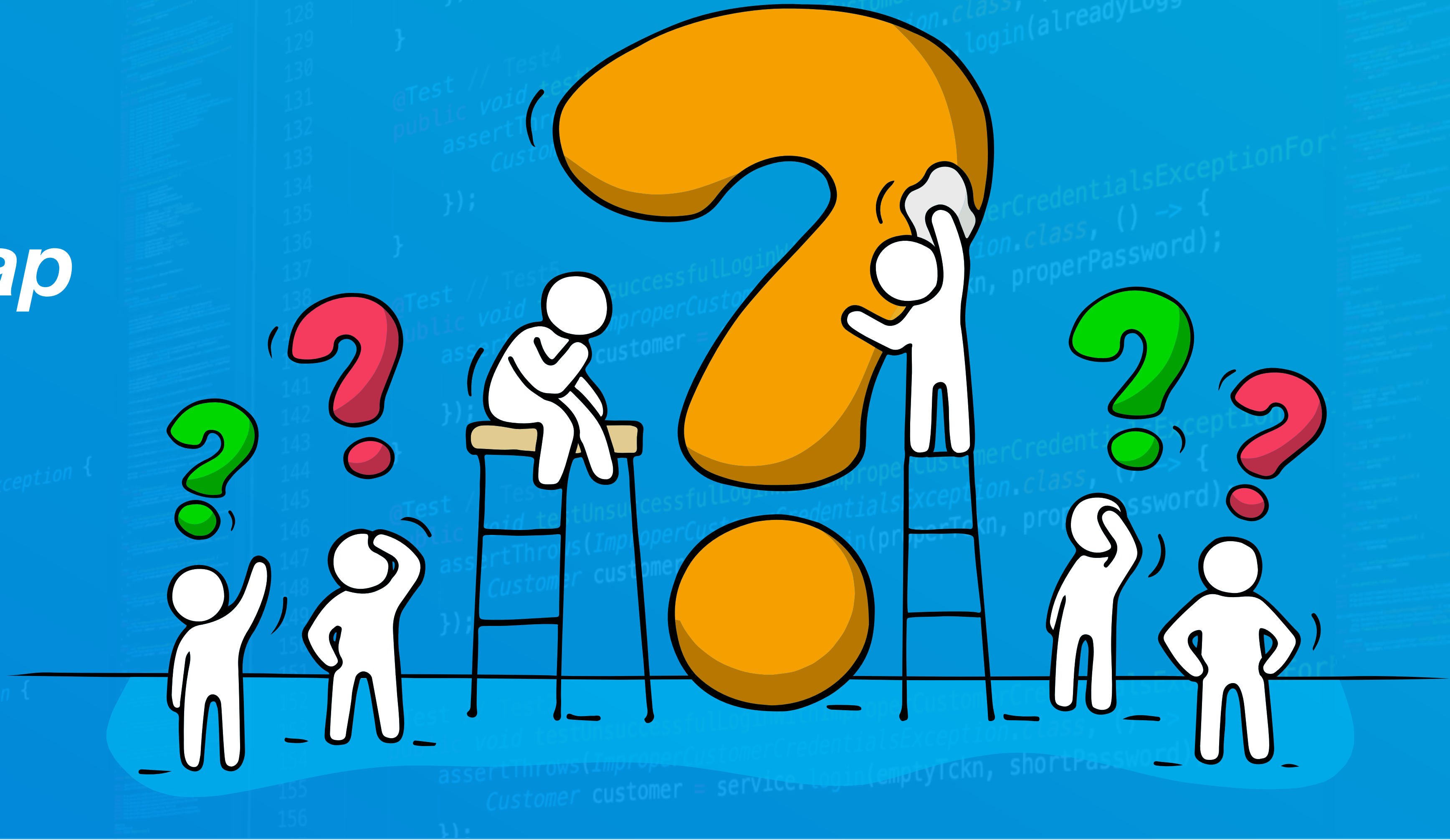


- Örneğin Java'nın `java.util` ve `java.sql` paketlerinde `Date` isimli bir sınıf vardır
- İki `Date` sınıfını da kullanmak istediğimizde kodumuza sadece birini dahil edip basit isimle, diğerini ise tam isimle kullanırız
- Eğer aynı isme sahip sınıflardan bir tanesi ile aynı pakette bulunuluyorsa, bu sınıfı dahil etmeye gerek olmadığından, basit isimle erişimde ona ulaşılır, diğerleri tam isimle kullanılır



- `sameName`
- `DateTest.java` kaynak kod dosyasında 3 farklı `Date` sınıfına ulaşım gösterilmiştir

Soru ve Cevap Zamanı!





static import

static import - I



- Bildiğiniz gibi, `static` üyelere sınıfın üzerinden ulaşılır
 - Dolayısıyla sınıf ismi her `static` üye erişiminde olmalıdır
- Java 5 ile birlikte bu konuda bir kolaylık getirildi
- Tüm `static` üyeler, değişken ya da metot, koda statik olarak dahil edilip sınıfın ismi olmadan kullanılabilir:

```
double area = Math.PI * r * r; // Old way
double area = PI * r * r; // New way after static import

System.out.println(Math.pow(2, 5)); // Old way
System.out.println(pow(2, 5)); // New way after static import
```

static import - II



- Bunun için **import static** cümlesi kullanılır
- **import static**'ten sonra sınıfın tam ismi ve üyenin ismi gelir
- **import static** ile hem değişkenler hem de metotlar koda dahil edilebilir

```
import static java.lang.Math.PI; // It reads static import
import static java.lang.Math.pow; // But written import static
import static java.lang.Math.*; // Imports all static members of Math

double area = Math.PI * r * r; // Old way
double area = PI * r * r; // New way after static import
System.out.println(Math.pow(2, 5)); // Old way
System.out.println(pow(2, 5)); // New way after static import
```

static import - III



- `static import` cümlesinde `*` kullanılabilir
 - Bu durumda sınıftaki tüm `static` yapılar, yani hem `static` değişkenler hem de `static` metotlar `import` edilir
- Dahil eden ve edilen yapılar aynı pakette olsalar bile, sınıfın tam ismi ve `static` üyenin ismi ya da `*` ile `static import` yapılmalıdır

StaticImport



- `staticImport.StaticImport`

static import - IV



- Bir kod parçasında, **static import** ile koda dahil edilen alan ya da metotlarla aynı isimde alan ya da metotlar varsa, bu durumda koddaki yapılara ulaşılacağını da unutmayın

```
import static java.lang.Math.*;
...

public static void main(String[] args) {
    double PI = 5.29;
    System.out.println(Math.PI + " " + PI);
    System.out.println(Math.pow(2, 10) + " " + pow(2,10));
}

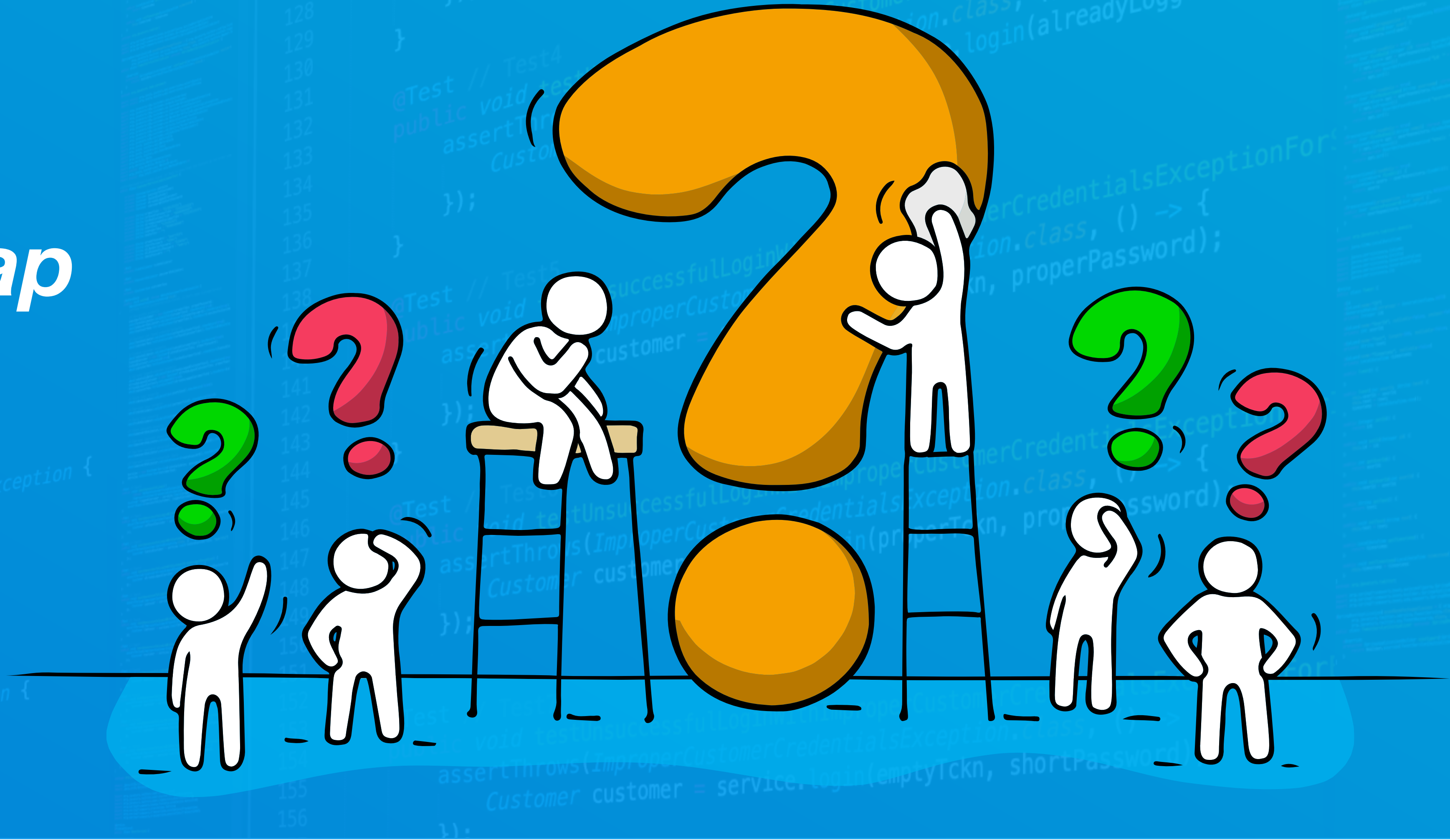
static double pow(double a, double b){
    return 1.0;
}
```

StaticImportCaution



- `staticImport.StaticImportCaution`

Soru ve Cevap Zamanı!





Java Kütüphanesinin Paketleri

Java'nın Paketleri - I



- Java'nın kütüphanelerinde pek çok paket vardır, bu paketler `java` ve `javax` ile başlar ve API'sinde ve kaynak kodunda görünür
- `java` şeklinde başlayan paketler *ayrılmıştır (reserved)*, bu şekilde paket oluşturulamaz
- Böyle bir pakette bir tip oluşturulup derlenebilir ama çalıştırılınca hata alınır:

```
Exception in thread "main" java.lang.SecurityException:  
Prohibited package name: java
```

Java'nın Paketleri - II



- Java'nın standart kütüphanesinin sık kullanılan paketlerinden bazıları şunlardır:
- `java.lang`: Java'nın en temel yapılarını barındırır
- `java.io`: Giriş-çıkış (IO) ve dosya işlemlerine yönelik yapılar bulunur
- `java.sql`: Java'nın veri tabanı iletişimi yapılarını barındırır
- `java.util`: Torba çerçevesi (collection framework) altyapısı, date ve time gibi araçsal yapılar bulunur

java.lang Paketi



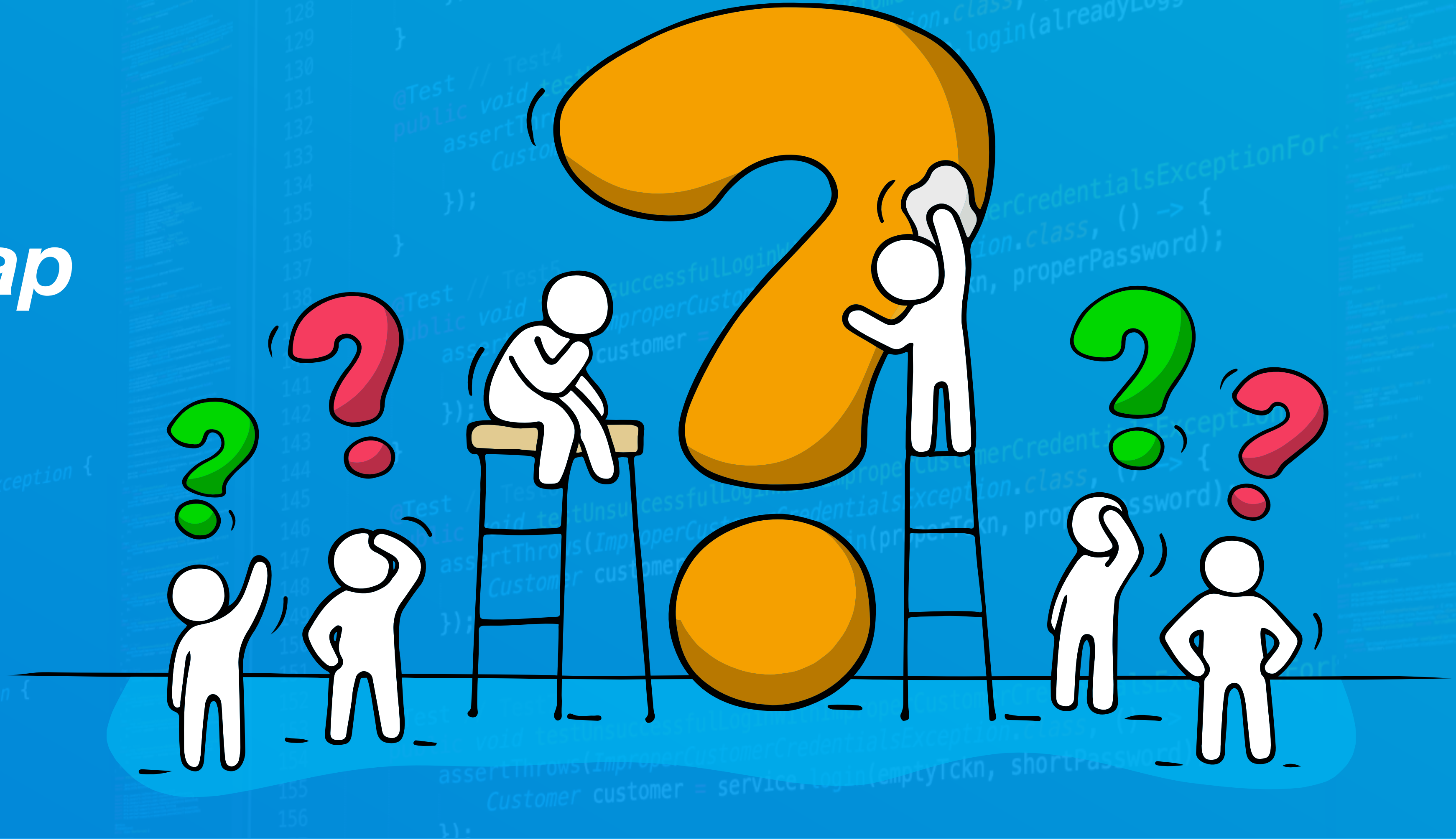
- `java.lang` paketi, dolayısıyla içindeki tüm yapılar, her Java kaynak koduna daima otomatik olarak dahil edilir
- `java.lang` paketini `String`, `System`, `Math` ve `IO` sınıflarından tanıyoruz.
- Diğer paketlerin yapılarına ulaşmak için onları `import` etmek gereklidir

Java Kütüphanesinin Paketleri



- <https://docs.oracle.com/en/java/javase/25/docs/api/index.html> adresindeki Java API'sinden paketleri inceleyin

Soru ve Cevap Zamanı!



Paket ve Dizin



Paketin Etkileri



- Paketler `java` kaynak kod dosyalarında belirtilir ama kaynak dosyalarını değil `class` dosyalarının içindeki sınıfları etkiler. Bu etki temelde iki türdür:
- Paket, o pakete sahip olan sınıfın ismini değiştirir, artık geçerli sınıf ismi *tam isimdir* (*fully qualified name*)
- Paketin `class` dosyasının ismine bir etkisi yoktur, değişen `class` dosyasının içindeki sınıfın ismidir
- Ayrıca paketler dizinlere karşı gelir ve bir `class` dosyası, paketine uygun dizinde olmalıdır

Paket ve Dizin - I



- Paketler, işletim sistemindeki dizinlere (directory, folder) karşı gelir
- Bir pakete sahip olan sınıfın `class` dosyasının, paketine karşı gelen dizinde olması şarttır
- Eğer paket iç içe paketlerden oluşuyorsa, benzer şekilde dizinler de iç içe dizinlerden oluşmalıdır
- İç içe geçen dizinler bir patika/yol (path) oluşturur

Paket ve Dizin - II



- Bu durum `java` ile biten kaynak kodu dosyaları için geçerli değildir
- Dolayısıyla paket daha çok bir çalışma zamanı yapısıdır ve `class` dosyalarını ilgilendirir
- Kaynak kodu dosyaları tamamen farklı bir dizinde olabilir
- Ama kolaylık olması için IDE'ler `java` uzantılı kaynak kod dosyalarını da paket yapısına uygun dizinlerde tutar

Örnek - I



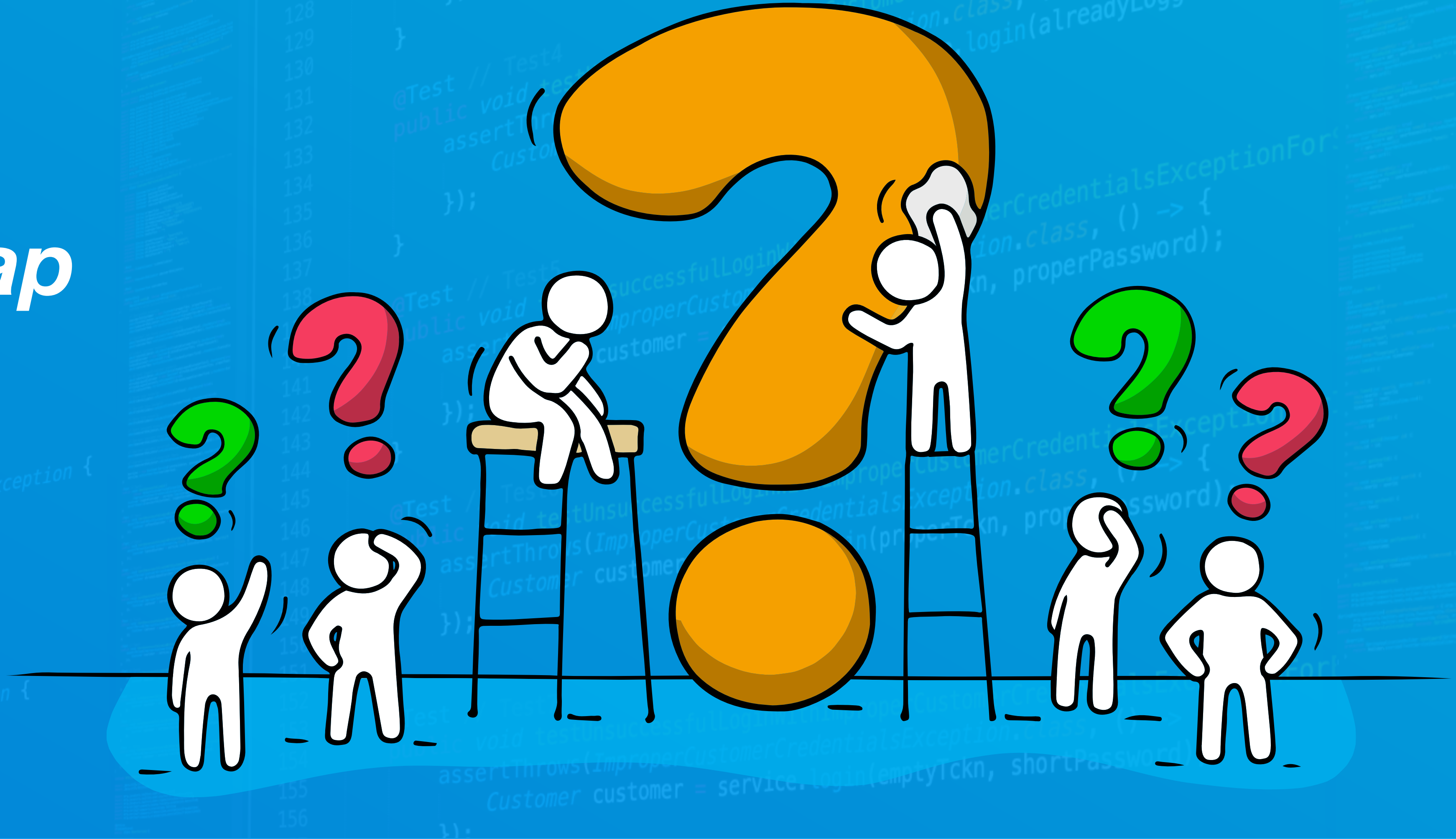
- Örneğin `ClassA` sınıfının paketi `org.javaturk.jp.ch21.packageA` ise
- Windows'ta bir yerde `org\javaturk\jp\ch21\packageA`
- MacOS'ta bir yerde `org/javaturk/jp/ch21/packageA`
- şeklinde bir dizin yapısı vardır ve `ClassA.class` dosyası da `packageA` dizininde demektir

Örnek - II



- Bu durumda
 - Windows'ta `org\javaturk\jp\ch21\packageA`
 - ya da MacOS'ta `org/javaturk/jp/ch21/packageA`
- şeklinde bir patika ya da yol (path) oluşur
 - İkisi de `packageA` dizininin patikasıdır

Soru ve Cevap Zamanı!



Derleme



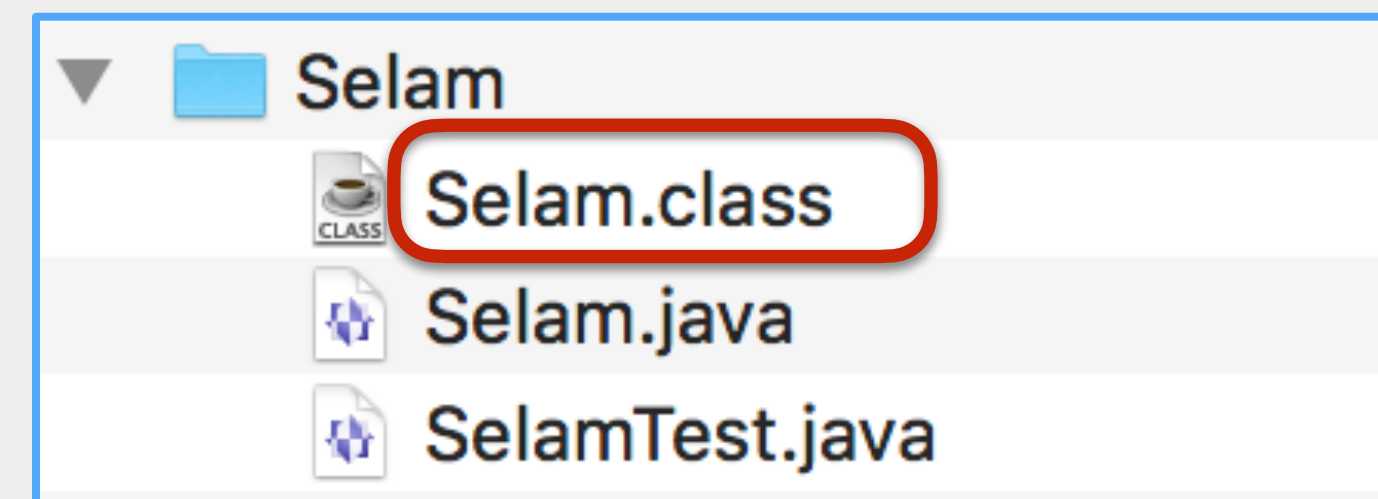
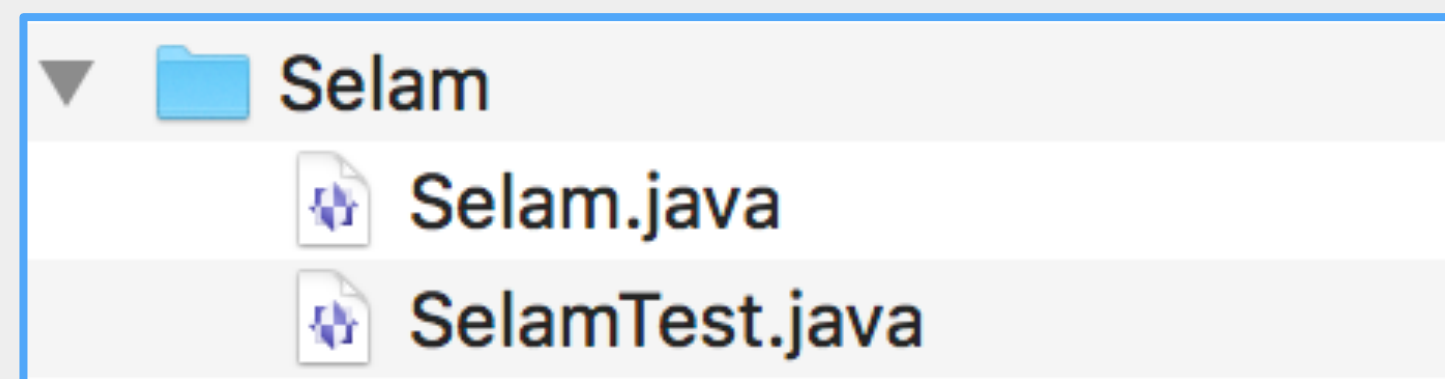
Kaynak Kodu Derlemek



- İsmi `.java` ile biten bir Java kaynak kodu `javac` ile derlendiğinde, kaynak koddaki her sınıfın `class` dosyası, kaynak kodların bulunduğu dizinde oluşturulur
- Pakete uygun dizinler yaratılmaz, onları elle oluşturmak gerekir

```
package org.javaturk.selam;  
...  
public class Selam{...}
```

```
../Selam > javac Selam.java
```



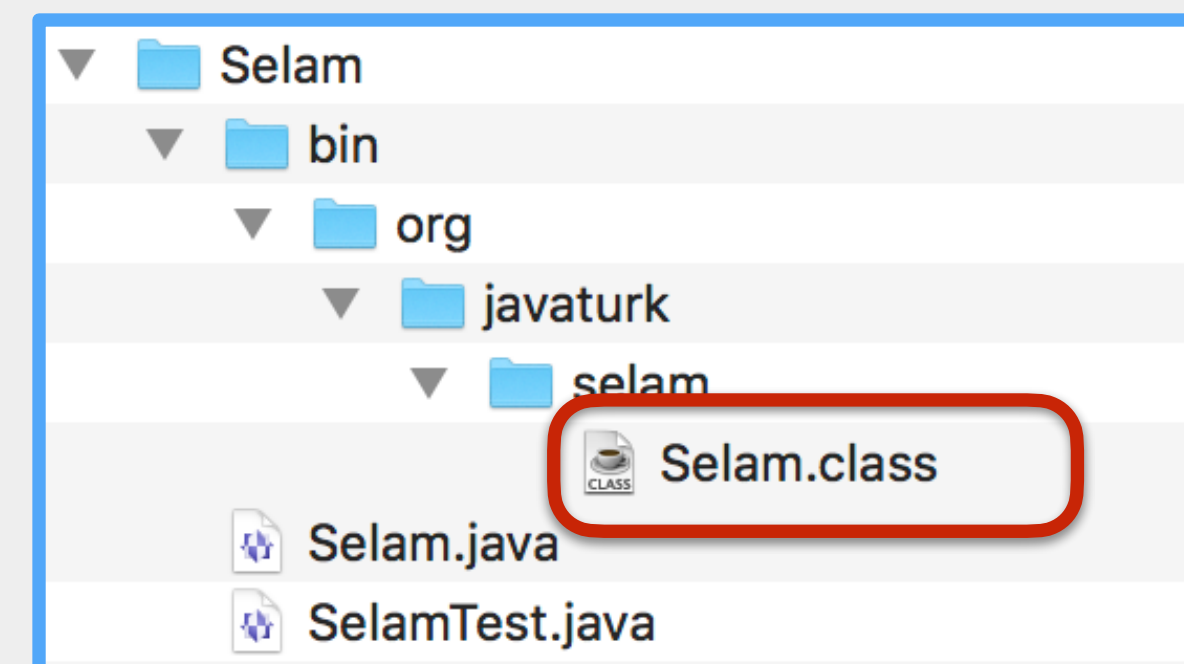
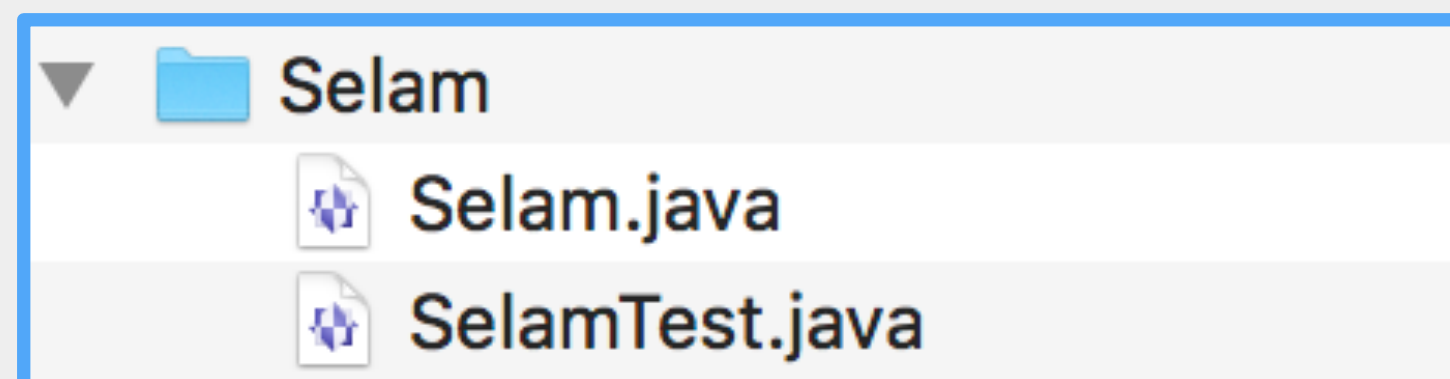
-d İle Bir Dizin Derlemek - I



- Ama derleyiciye `d` seçeneği (flag, option) ile bir dizin geçilirse, derleyici bu dizinde, `d` ile belirtilen dizin dahil olmak üzere, pakete uyan tüm dizinleri oluşturur ve `class` dosyasını belirtilen dizine koyar
- Derleyici seçenekleri `-` ile belirtilir dolayısıyla `-d` olarak kullanılır

```
package org.javaturk.selam;  
...  
public class Selam{...}
```

```
../Selam > javac -d ./bin Selam.java
```



-d İle Bir Dizin Derlemek - II



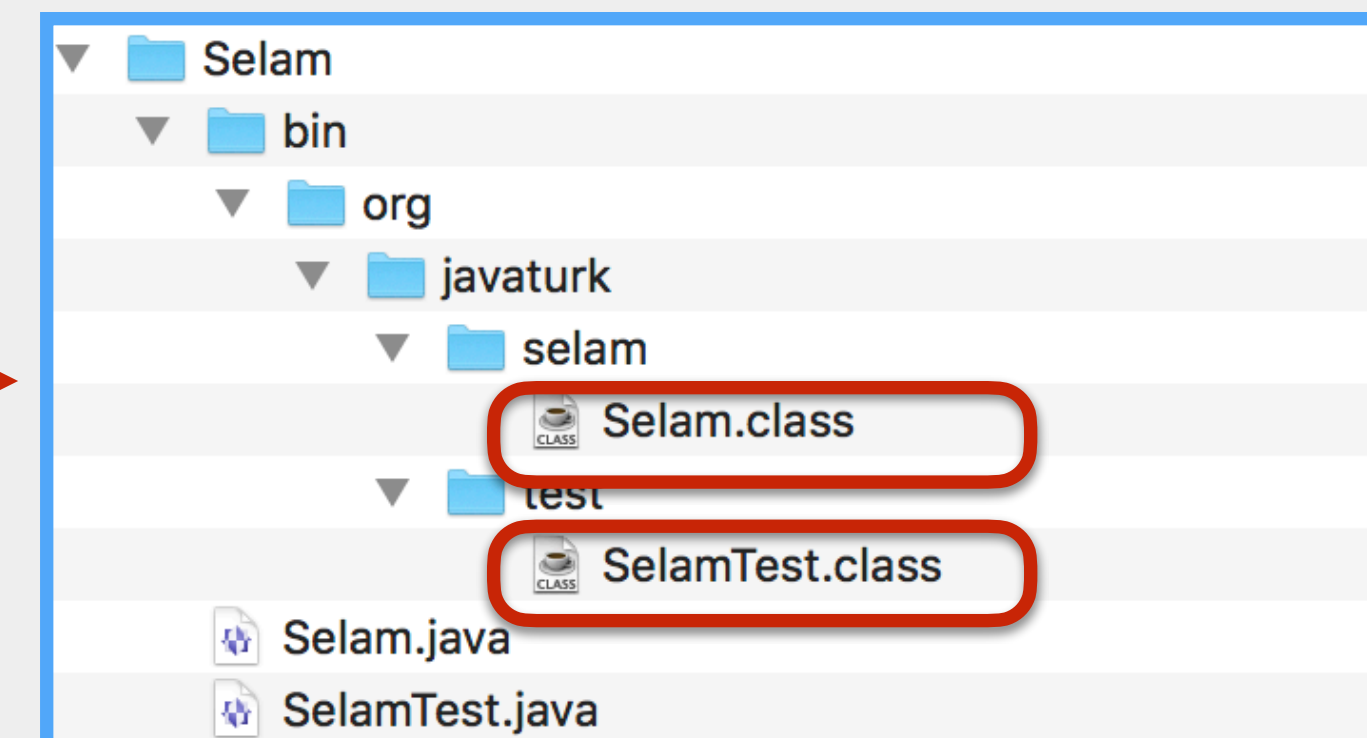
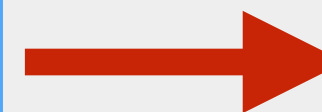
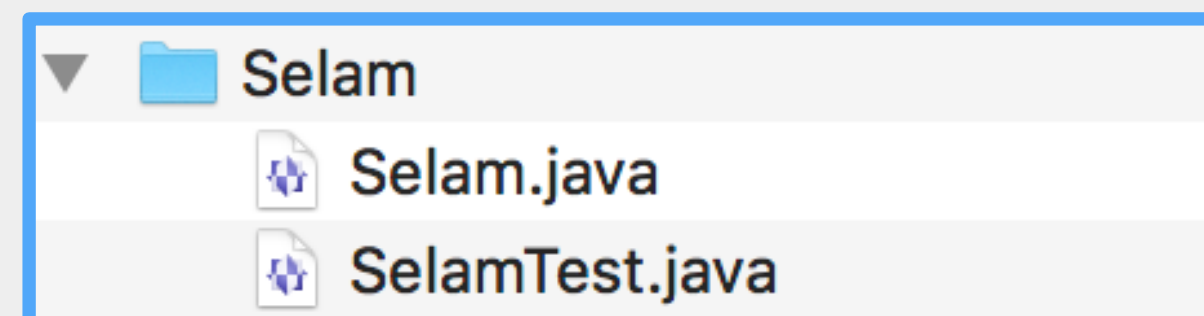
- Aynı dizindeki birden fazla kaynak dosyası hep birlikte, isimleri tek tek sıralanarak derlenebileceği gibi `*.java` kullanarak da derlenebilir
- `-d` seçeneği ile pakete uygun dizin yapısı oluşturulur

```
package org.javaturk.selam;  
...  
public class Selam{...}
```

```
package org.javaturk.test;  
...  
public class SelamTest{...}
```

```
.../Selam > javac -d ./bin Selam.java SelamTest.java
```

```
.../Selam > javac -d ./bin *.java
```



Selam ve SelamTest



- İkinci bölümün kod örneklerinden olan `Selam.java` ve `SelamTest.java` sınıflarını, komut satırında `-d` kullanmadan ve kullanarak derleyip aradaki farkı gözlemleyin

Varsayılan Paketin Dizini



- İsimsiz yani varsayılan paket, dizin olarak içinde bulunulan dizine işaret eder
- Varsayılan pakete sahip sınıf derlendiğinde herhangi bir dizin yapısı oluşturulmaz
- Çünkü paket yoksa dizin ismi de yok demektir

Kısa Kaynak Kod Dosyalarını Derleme



- Kısa kaynak kod dosyaları bir pakete sahip olamazlar ama derlendiklerinde içinde bulundukları `java` uzantılı dosyanın ismine sahip bir `class` dosyası oluşur
- Ve yine bir pakete sahip olamadıklarından dolayı derlendiklerinde oluşan `class` dosyası herhangi bir yerde olabilir

Source-file Mode ve Script Çalıştırma



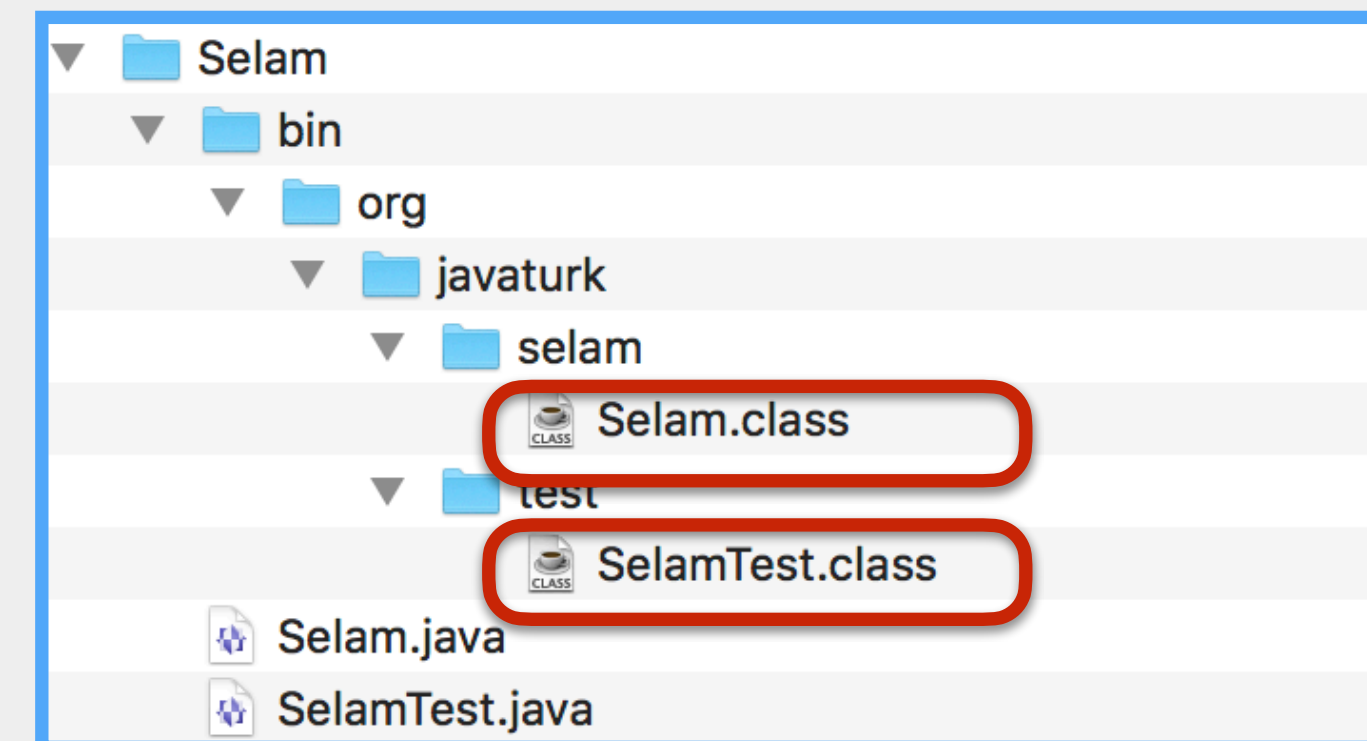
- Java 11 ile gelen ve sonra geliştirilen bir özellik, **source-file mode** çalıştırmada `java` komutuna `main` metodun içinde olduğu dosyanın ismi geçer
- Aynı özellik, sadece tek bir dosyadan oluşan kaynak koda sahip programları, Unix, Linux ve MacOS sistemlerinde **script (betik)** olarak da çalıştırmak mümkün hale geldiğini de öğrenmiştik

Çalıştırma

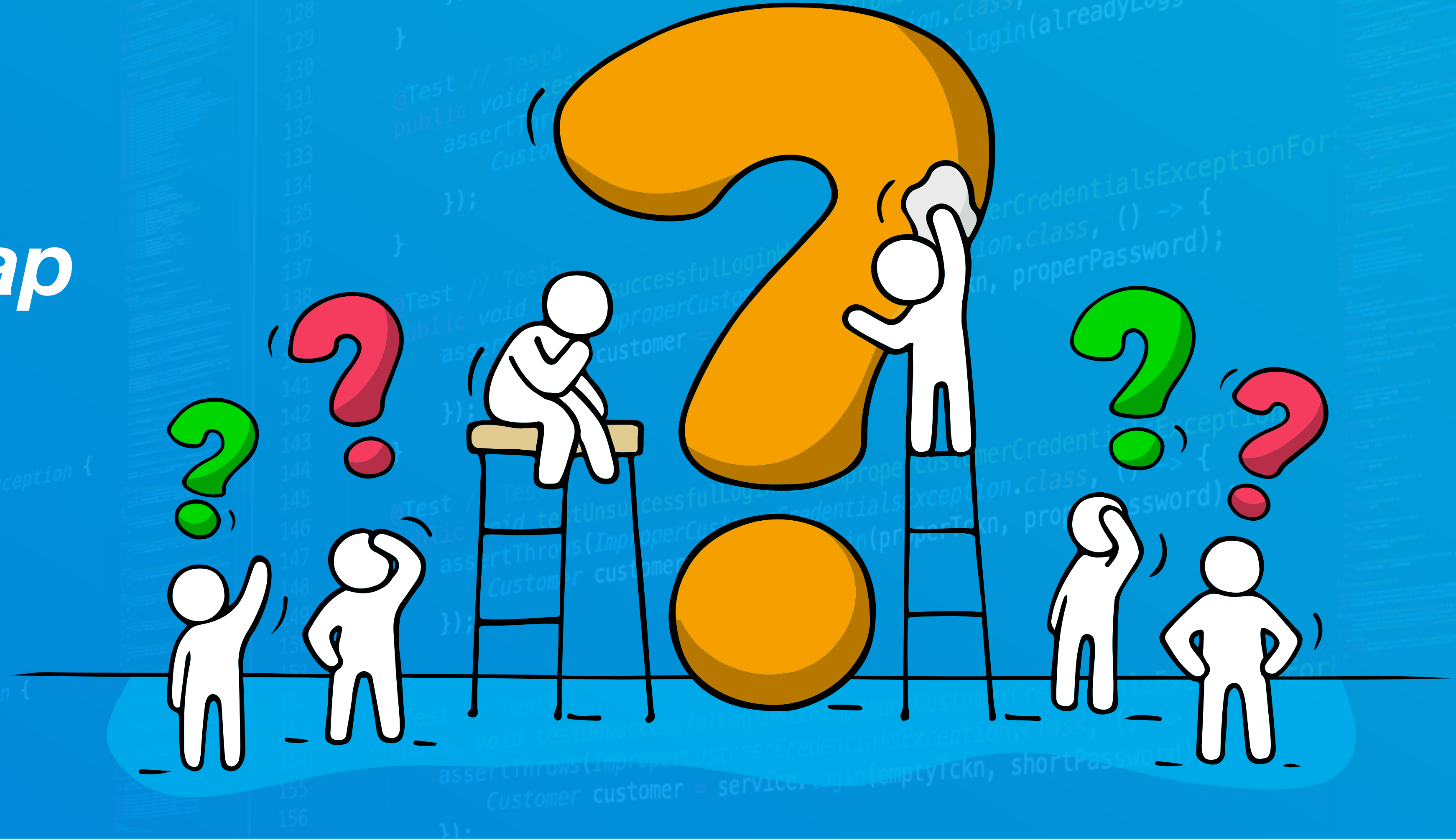


- Geriye çalıştırmak kaldı
- Eğer `java org.javaturk.test.SelamTest` komutuyla `SelamTest` sınıfını çalıştırmaya kalkarsak hata alırız
- Bu bir `CLASSPATH` problemidir ve bir sonraki bölümün konusudur

```
.../Selam > java org.javaturk.test.SelamTest
```



Soru ve Cevap Zamanı!



Ödevler





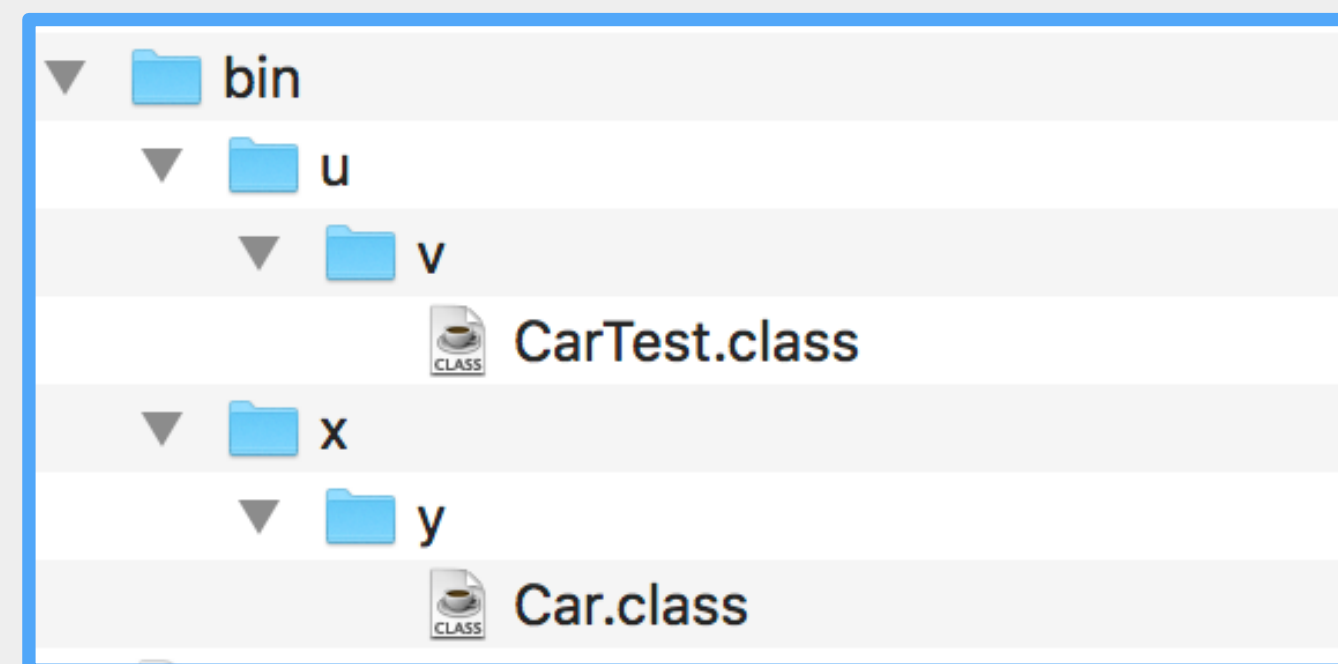
1. Eğitimde daha önce var olan `Car` ve `CarTest` sınıflarına, IDE kullanmadan, gerekli `package` ve `import` cümlelerini koyarak, aşağıdaki gibi bir paket yapısına sahip olmalarını sağlayın

```
package x.y;  
...  
public class Car{...}
```

```
package u.v;  
import x.y.Car;  
...  
public class CarTest{...}
```



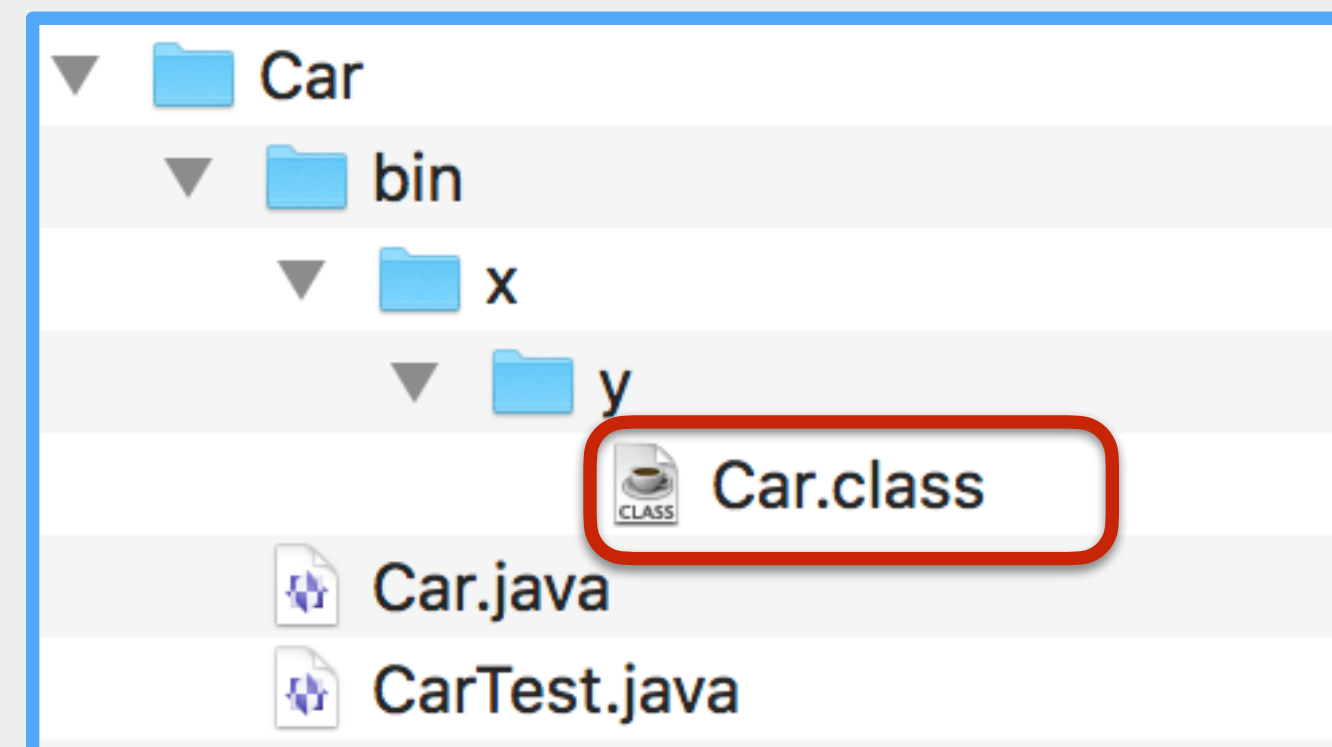
- Sonra `Car.java` ve `CarTest.java` sınıflarını `-d` seçeneği ve `*.java` ile derleyin ve `class` dosyalarının `bin` dizini altında paket yapısına uygun bir dizine sahip olmalarını sağlayın





- Sonra `bin` dizinini silin
- Önce `Car.java`'yı `-d` seçeneği ile derleyin ve paket yapısına uygun bir dizin yapısı oluşmasını sağlayın
- Sonra benzer şekilde `CarTest.java` sınıfını `-d` seçeneği ile derleyin

- Ne sonuç aldınız?





- Bu bir `CLASSPATH` problemidir ve bir sonraki bölümde bunu ele alacağız



2. Bir önceki uygulamadaki `Car` ve `CarTest` sınıflarını, `CarTest.java` kaynak dosyasında birleştirin ve `java` ile doğrudan çalıştırın
 - a. Aynı dosyayı daha sonra shebang ile `car` adlı bir dosyaya kaydedin ve script olarak çalıştırın

```
public class CarTest{...}  
class Car{...}
```

Bundan Sonra?



Bundan Sonra?



- Bundan sonra, bu bölümün devamı niteliğindeki `CLASSPATH` ele alınacaktır

Bölüm Sonu

Soru ve Cevap Zamanı!

